Chapter 7 n-step Bootstrapping

寻找最佳的“平衡点”

本章介绍的 **n步时序差分 (n-step TD)** 是一种将 **蒙特卡洛方法 (Monte Carlo, MC)** 和 **单步时序差分方法 (one-step TD)** 统一在一个更通用框架下的技术。通过调整步数 n，你可以根据任务的需求，在这两种极端方法之间平滑过渡，寻找最佳平衡。

在理解 n 步自举法之前，需要先复习两个极端的估值方法。假设你要估算从你家开车去公司的总时间：

- **蒙特卡洛方法 (Monte Carlo, MC):** 你一路开到公司（走完一个完整的 Episode），把一路上的真实耗时加起来作为最终结果来更新你的初始估测。
  - *特点：* 看得最远，没有偏差（都是真实数据），但方差很大（每次路上遇到的红绿灯、堵车情况都不同）。
- **单步时序差分 (One-step TD):** 你只开到下一个路口（只走 1 步），记录这段真实时间，再加上你对“从下一个路口到公司”的估算时间，用这个总和来更新初始估测。
  - *特点：* 结合了真实反馈和已有经验（这叫**自举 Bootstrapping**），更新快，方差小，但一开始可能有偏差（因为你的估算可能不准）。

n 步自举法就是这两者的**通用框架**和**桥梁**。它允许你往前开 n 个路口（收集 n 步的真实耗时），然后再基于第 n 个路口估算剩下的时间。
• 当 $n = 1$ 时，它就是 One-step TD。
• 当 $n = \infty$（或者大到终点）时，它就是 Monte Carlo。
通过调整 n 的大小，你可以平滑地在两者之间切换，找到最适合当前任务的平衡点。

n-step TD Prediction

在估计某个策略 π 下的状态价值函数 V^π 时，我们需要计算一个**目标值 (Target)**，让我们的估计值 $V(s_t)$ 不断逼近这个目标。

基础符号：

- t: 当前时间步。
- T: 回合 (Episode) 结束的时间步（总长度）。
- γ: 折扣因子 (Discount rate)，取值在 0 到 1 之间，表示未来的奖励在当前看来要打几折。
- r_{t+1}: 在时间步 t 执行动作后，在 $t+1$ 时刻收到的真实奖励。
- $V(s_t)$: 我们当前对状态 s_t 价值的估计。

1. 蒙特卡洛的目标值 (全量回报 Full Return)

一直累加真实奖励直到回合结束：

$$g_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{T-t+1} r_T$$

2. 单步 TD 的目标值 (单步回报 One-step Return)

只看 1 步真实奖励，剩下的用估计值代替（公式 7.1）：

$$g_{t:t+1} \triangleq r_{t+1} + \gamma V^t(s_{t+1})$$

3. 两步 TD 的目标值 (两步回报 Two-step Return)

看 2 步真实奖励，剩下的用估计值代替（公式 7.2）：

$$g_{t:t+2} \triangleq r_{t+1} + \gamma r_{t+2} + \gamma^2 V^{t+1}(s_{t+2})$$

*** 注意：* 第二步的估计值前面乘了 γ^2，因为这是在两步之后的价值，折旧了两次。

4. n步 TD 的目标值 (n步回报 n-step Return)

推广到任意正整数 n（公式 7.3）：

$$g_{t:t+n} \triangleq r_{t+1} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n V^{t+n-1}(s_{t+n})$$

$$= \left(\sum_{i=1}^{n} \gamma^{i-1} r_{t+i} \right) + \gamma^n V^{t+n-1}(s_{t+n})$$

公式拆解： 前半部分括号里是前 n 步真实奖励的折现总和；后半部分是第 n 步状态的估计价值，乘以 γ^n 折现到现在。

这个策略是绝对固定的，这个算法的本质是Prediction (预测/评估)，也就是在每一个S的评估未来预期V

Algorithm 7.1 n-step TD prediction

```
Initialize V(s) for each s ∈ S arbitrarily
Set V(s) = 0 for each terminal state s
for each episode do
    Set T ← ∞
    Choose s_0 arbitrarily
    for each time step t = 0, 1, ... do
        if t < T then                                   ▷ terminal not reached yet
            Choose an action a_t following π(·|s_t)
            Take action a_t and observe the reward r_{t+1} and the next state s_{t+1}
            If s_{t+1} is terminal, set T = t + 1
        end if
        Let τ = t − n + 1
        if τ > 0 then                                   ▷ To update value for s_τ
            Let g = Σ_{i=1}^{min(n,T−τ)} γ^{i−1} r_{τ+i}   ▷ Compute n-step return
            if τ + n < T then
                Let g ← g + γ^n V(s_{τ+n})
            end if
            Update V(s_τ) ← V(s_τ) + α(g − V(s_τ))
        end if
        Exit the loop when τ = T − 1
    end for
end for
return V
```

主要在于：**“延迟更新 (Delayed Update)”** 机制

“延迟更新”： 假设 $n = 3$。你在时刻 $t = 0$ 时，不能立刻计算 3 步回报并更新状态 s_0 的价值。为什么？因为你还没拿到未来 3 步的奖励！你必须继续往前走，直到时间走到 $t = 2$（经历了 3 次状态转移），你收集齐了 r_1, r_2, r_3，这时你才能“翻旧账”，回过头去更新 s_0。

算法步骤逐行解析：

1. **初始化：** 随便给每个状态一个初始价值 $V(s)$，但终结状态的值必须是 0。

**注意，这里的随机选择起点s0，为了全面扫描地图，多进行几次，从而更快地把所有状态的价值 V 都算

2. **开始循环回合：** 设 $T = \infty$（还没找到终点），随便选个起点 s_0。
3. **时间步 t 开始推进 ($t = 0, 1, 2 \ldots$)：**
   - 如果还没到终点 ($t < T$)：按照策略采取动作，观察拿到奖励 r_{t+1} 和新状态 s_{t+1}。如果碰到了终点，就记录下终点的时刻 $T = t + 1$。
   - **计算要更新的目标时间戳 τ：** $\tau = t - n + 1$。
     - *举例：* $n = 3$。当时间走到 $t = 2$ 时，$\tau = 2 - 3 + 1 = 0$。这意味着"当时间走到第3步时，我们终于收集齐了数据，现在要开始更新第0步的状态 s_0 啦！"

为什么要等？：假设 $n = 3$。你在 $t = 0$ 时处于 S_0。你想用 $n = 3$ 的公式更新 S_0。公式需要 R_1, R_2, R_3。但是！现在时间才 $t = 0$，你根本不知道两步之后的 R_2, R_3 是多少！你只能把 S_0 的更新先**搁置（存进缓存/小本本）**，继续往下走。

核心推演：以 $n = 3$ 为例，带你走一遍循环！

假设你正在走迷宫，$n = 3$。我们用一个表来记录时间的流逝：

- **初始：** 你在起点 S_0。当前时间 $t = 0$。
- **走第 1 步 ($t = 0$ 时)：**
  - 做动作 A_0，拿到 R_1，走到 S_1。
  - 计算 $\tau = t - n + 1 = 0 - 3 + 1 = \mathbf{-2}$。
  - **$\tau < 0$ 的处理（回答你的问题 8）：** 代码里写了 `if tau >= 0` 才执行更新。所以 $\tau = -2$ 时**什么都不更新**。我们只是把 (S_0, R_1, S_1) 记在小本本上。继续走！
- **走第 2 步 ($t = 1$ 时)：**
  - 做动作 A_1，拿到 R_2，走到 S_2。
  - 计算 $\tau = 1 - 3 + 1 = \mathbf{-1}$。
  - 依然 $\tau < 0$，**什么都不做**，继续记账。
- **走第 3 步 ($t = 2$ 时)：**
  - 做动作 A_2，拿到 R_3，走到 S_3。
  - 计算 $\tau = 2 - 3 + 1 = \mathbf{0}$。
  - **叮！$\tau \geq 0$ 了！这说明凑齐了 3 步的数据！**
  - **开始"秋后算账"：** 我们终于可以更新 S_τ，也就是 S_0 了！
  - 收集真实奖励：$g = R_1 + \gamma R_2 + \gamma^2 R_3$。
  - **第 n 步估计值自举（回答你的问题 7）：** 因为只看了 3 步，距离终点还不知道有多远，所以加上此时 S_3 的**模型预估值**：$g \leftarrow g + \gamma^3 V(S_3)$。这就像"我真实走了3步，后面的路我懒得走了，我按目前对第3步位置的经验估算一下"。这叫自举。
  - 拿着这个目标值 g，正式更新 $V(S_0)$。
- **走第 4 步 ($t = 3$ 时)：**
  - 做动作 A_3，拿到 R_4，走到 S_4。
  - 计算 $\tau = 3 - 3 + 1 = \mathbf{1}$。
  - 更新 S_1！用真实的 (R_2, R_3, R_4) 加上估测的 $V(S_4)$。

   - **如果 $\tau \geq 0$ (说明已经凑够了 n 步数据，可以更新了)：**
     - 先计算前 n 步的累积真实奖励 g。
     - 如果没到回合终点 ($\tau + n < T$)，加上第 n 步的估计值 $\gamma^n V(s_{\tau+n})$ 进行自举。
     - **执行更新：** 使用标准的增量更新公式 $V(s_\tau) \leftarrow V(s_\tau) + \alpha(g - V(s_\tau))$。（$\alpha$ 是学习率，相当于每次向目标值迈多大的步子）。
   - **结束条件：** 当 $\tau = T - 1$ 时（即回合里所有的状态都完成了属于自己的 n 步延迟更新），退出循环。

注意

- **问题：** 如果设定的步数 $n = 5$，但我只走了 3 步游戏就结束了（撞墙了或者通关了），怎么办？
- **答案：** 当 $t + n \geq T$（即预计的 n 步已经超出了回合总长度）时，缺少的那些奖励项以及最后的估计价值全部**视作 0 (taken as zeros)**。
  这种情况下，n 步回报 $g_{t:t+n}$ 就自然退化成了真实的蒙特卡洛全量回报 g_t。当 n 设置为无穷大时，这个算法就完全等同于蒙特卡洛方法。

我想问：1.对于一个状态st，做出的一个动作是at还是at+1？ 以及*n*-step TD prediction 目的是什么？随便选一个起点是S0是什么意思？举个例子分析，难道不是有始有终的嘛？以及在这个算法中，策略都是固定的嘛？不涉及策略的更新？只有对这个策略下的每个状态的价值分析？那分析出来之后干啥？目标时间戳是什么？为什么非要走到那里才进行更新状态？更新是比如n=3，是每个s1，2，3都进行，以及第n的估计值自举是什么意思，那小于时间戳的怎么进行自举？不是变成蒙特卡洛的期望了嘛？同时怎么计算小于时间戳的状态V？难道只能从等于时间戳的开始嘛？那怎么计算呢？还是跑多次？然后求？举个例子分析

n步 Sarsa 算法 （n-step Sarsa）

从 V 值到 Q 值

在之前的 n 步 TD 预测中，我们只关心**状态价值** $V(s)$（即评估“处于某个状态”有多好）。但是，如果我们想让智能体自己做决策（找出最优策略），光知道状态好坏是不够的，我们必须知道**动作价值** $Q(s, a)$（即评估“在某个状态下采取某个具体动作”有多好）。

n步 Sarsa (n-step Sarsa) 就是将 n 步 TD 预测与 Sarsa 算法结合起来，得到的一种**同策略 (on-policy)** 学习算法。

- **核心思想：** 把基于 V 值的 n 步更新规则，扩展到基于 Q 值的状态-动作对 (state-action pairs) 上，并使用 ε-贪心 (ε-greedy) 策略来一边探索一边更新。

通俗举例： 假设你在下国际象棋。

- **评估 V 值 (n-step TD)：** 你往前推演了 n 步，评估的是“n 步之后**这个棋局**对我有多利”。
- **评估 Q 值 (n-step Sarsa)：** 你往前推演了 n 步，评估的是“n 步之后，面对那个棋局，我**决定走的下一步特定棋**到底好不好”，以此来反推你现在正准备走的这一步棋的价值。

核心的公式：

1. 状态价值的 n步 TD 更新公式 (公式 7.4)

$$V^{t+n}(s_t) \triangleq V^{t+n-1}(s_t) + \alpha(g_{t:t+n} - V^{t+n-1}(s_t))$$

• 这是对上一节的总结：用算出的目标值 $g_{t:t+n}$ 减去旧的估计值，乘以学习率 α，加到旧估计值上完成状态 s_t 的更新。

2. 动作价值的 n步更新法则 (公式 7.5)

$$Q_{t+n}(s_t, a_t) = Q_{t+n-1}(s_t, a_t) + \alpha(g_{t:t+n} - Q_{t+n-1}(s_t, a_t))$$

• **符号解析：** 我们现在更新的主体变成了 $Q(s_t, a_t)$。只有当前时间步 t 对应的 (s_t, a_t) 会被更新，其他的状态-动作对在这一步保持不变。

3. n步 Sarsa 的目标值 / n步回报 (公式 7.6)

$$g_{t:t+n} \triangleq r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n Q_{t+n-1}(s_{t+n}, a_{t+n})$$

- **公式拆解：** 前半部分依然是 n 步内的真实打折奖励总和。**重点在最后一项**：我们用来“垫底”（自举 Bootstrapping）的值，变成了第 n 步的状态 s_{t+n} 和我们在那一步**实际选择的动作** a_{t+n} 对应的 Q 值。
- **边界情况：** 和之前一样，如果 $t + n \geq T$（超过了回合结束时间），缺失的项和最后的 Q 值都当成 0 处理。

Algorithm 7.2 n-step Sarsa

```
Initialize Q(s, a) for each s ∈ S and a ∈ A(s) arbitrarily
Initialize π to be ε-greedy w.r.t. Q
Set V(s) = 0 for each terminal state s
for each episode do
    Set T ← ∞
    Choose s_0 arbitrarily
    for each time step t = 0, 1, ... do
        if t < T then                                          ▷ terminal not reached yet
            Choose an action a_t following π(·|s_t)
            Take action a_t and observe the reward r_{t+1} and the next state s_{t+1}
            If s_{t+1} is terminal, set T = t + 1
        end if
        Let τ = t − n + 1
        if τ > 0 then                                          ▷ To update value for (s_τ, a_τ)
            Let g = Σ_{i=1}^{min(n,T−τ)} γ^{i−1} r_{τ+i}       ▷ Compute n-step return
            if τ + n < T then
                Let g ← g + γ^n Q(s_{τ+n}, a_{τ+n})
            end if
            Update Q(s_τ, a_τ) ← Q(s_τ, a_τ) + α(G − Q(s_τ, a_τ))
            Update π to be ε-greedy w.r.t. Q
        end if
        Exit the loop when τ = T − 1
    end for
end for
return V
```

算法解析（n step Sarsa）

算法 7.2 的整体逻辑（比如 $\tau = t - n + 1$ 的延迟更新机制）与 7.1 非常相似。它们之间的 **两个核心区别**：

- **更新对象不同：** n 步 Sarsa 维护和更新的是状态-动作对的 Q 值表，而不是状态的 V 值表。
- **策略在动态进化 (Control 过程)：** 在算法 7.1 中，策略 π 是固定的（只是评估它）。但在 7.2 中，有一行伪代码：`Update π to be ε-greedy w.r.t. Q`。这意味着每次 Q 值更新后，策略 π 都会立刻调整，确保智能体总是倾向于选择当前看起来 Q 值最大的动作（保留 ε 的随机性用于探索）。

n步期望 Sarsa (n-step Expected Sarsa)

在标准 Sarsa 中，公式末尾用来评估的 $Q_{t+n-1}(s_{t+n}, a_{t+n})$ 依赖于你最后一步**实际抽样到**的具体动作 a_{t+n}。因为策略带有随机性（比如 ε-贪心），如果最后一步运气不好抽到了一个很差的探索性动作，整个目标值就会产生剧烈波动（高方差）。
为了解决这个问题，引入了 **期望 Sarsa (Expected Sarsa)**。

1. 期望 Sarsa 的目标值 (公式 7.7)

$$g_{t:t+n} \triangleq r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n \bar{V}_{t+n-1}(s_{t+n}), \tag{7.7}$$

• 注意最后一点变成了 $\bar{V}$ (期望近似值)，而不是具体的 Q。

2. 期望值的计算 (公式 7.8)

$$\bar{V}_t(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) Q_t(s,a).$$

• **公式拆解：** 我们把状态 s 下所有可能的动作 a 遍历一遍。把每个动作的 Q 值，乘以策略 π 选择该动作的概率，然后求和。这就是数学期望。跟之前的求期望的 Sarsa 类似。

通俗举例： 假设你开了家店，想评估当前地段的商业价值。

- **标准 Sarsa：** 随便拉一个路人问他愿不愿意花钱（抽样）。如果碰巧拉到个铁公鸡，你会误以为地段很差。
- **期望 Sarsa：** 你调查了所有路人的消费倾向分布（概率），算出这个地段**平均**每个人会花多少钱（期望）。显然，后者评估出的价值更稳定、更靠谱。

n步树状备份算法 （n-step Tree Backup Algorithm）

引入动机

- **痛点 1：** 标准的 n 步 TD 预测完全依赖一条**实际采样**出来的动作序列。如果在某一步由于随机探索选了一个极其糟糕的动作，整条路径的估值都会被带偏（在实践中会导致**高方差**）。
- **痛点 2：** 前一节学的 **期望 Sarsa (Expected Sarsa)** 缓解了这个问题，但它**只在最后一步**（第 n 步）计算了期望值。它前面的 $n-1$ 步依然是“一条道走到黑”，只看实际选中的动作。

既然算期望能降低方差，我们能不能**在每一步都计算期望收益**，而不是仅仅依赖选中的那一个动作？
沿着这个思路，就诞生了n**步树状备份算法**。

核心思想：主干与枝叶 (The Tree Structure)
在这个算法中，每走一步（时间步 t），我们不仅考虑**实际选中的动作** A_t 和它的奖励，还要考虑那些**没有被选中的动作**。

把这个过程想象成一棵树：

- **主干 (Selected actions)：** 你实际走的路径，它会把你带向真实的下一个状态。
- **枝叶 (Unselected actions)：** 那些你在路口没有选择的动作。我们把它们作为计算中的“叶子节点”，直接用现有的 Q 值模型去**自举 (bootstrap)** 估算它们的价值。

通俗举例： 假设你在玩走迷宫，每到一个路口有三扇门。

- *之前的算法：* 你随便进了一扇门，对其他两扇门看都不看，连续穿过 n 扇门后才评估。
- *树状备份：* 你走进一扇门（主干），但在进门前，你**往另外两扇没选的门里偷偷张望了一下**（用模型评估这些“错过的风景”），并把这些没选的门的价值，按照你原本可能选择它们的概率，加权算进当前路口的总价值里。

核心公式

1. 单步树状回报 (One-step tree-backup return)
当 $n = 1$ 时，这个算法就退化成了我们之前学的期望 Sarsa（公式 7.9）：

$$g_{t:t+1} \triangleq r_{t+1} + \gamma \sum_a \pi(a|s_{t+1}) Q_t(s_{t+1}, a).$$

• *解释：* 拿到第一步的真实奖励 r_{t+1}，加上下一步所有可能动作 a 的 Q 值的加权平均（即期望）。

你往前走了一步，拿到了真实的奖励 r_{t+1}。到了新的路口 s_{t+1} 时，你不只看下一步实际选了啥，而是**环顾四周**，把所有可能动作 a 的价值 Q，按照你会选它们的概率 π 求个平均值（期望），然后再打个折 (γ) 加起来。

---

2. 2 步树状回报的递归公式（Two-step tree-backup return）（公式 7.12）—— 核心重点

为了推广到 n 步，我们看这个极其优雅的递归公式：

$$g_{t:t+2} \triangleq r_{t+1} + \gamma \sum_{a \neq a_{t+1}} \pi(a|s_{t+1}) Q_{t+1}(s_{t+1}, a) + \gamma \pi(a_{t+1}|s_{t+1}) g_{t+1:t+2}, \tag{7.10}$$

where

$$g_{t+1:t+2} = r_{t+2} + \gamma \sum_{a} \pi(a|s_{t+2}) Q_{t+1}(s_{t+2}, a).$$

这个目标值由 **3 个部分** 拼装而成，我们逐个拆解：

1. r_{t+1} **(眼前利益)：** 你刚走完第一步拿到的真实奖励。
2. **中间的求和项 (错过的枝桠)：** 注意求和符号下面是 $a \neq a_{t+1}$。这代表在路口 s_{t+1} 时，你**没有选**的那些动作。既然没选，我们就直接用现成的 Q 表去估算它们的价值，并乘以你本来可能选它们的概率。
3. **最后一项 (继续延伸的主干)：** a_{t+1} 是你**真正选了**的动作，它带你走向了更深的未来。所以它的价值不能简单用 Q 表糊弄，而要用它后续更精确的回报 $g_{t+1:t+2}$ 来代替。
   - **重点来了：** 为什么要乘以 $\pi(a_{t+1}|s_{t+1})$？因为就算你实际走了这条路，但如果你的策略 π 告诉你选这条路的概率只有 10%，那么这条路后续不管发生什么惊天动地的事，对你当前总价值的影响都要**缩水到 10%**。

$$\begin{aligned} g_{t:t+2} = {} & r_{t+1} + \gamma \pi(a_{t+1}|s_{t+1}) r_{t+2} \\ & + \gamma \sum_{a \neq a_{t+1}} \pi(a|s_{t+1}) Q_{t+1}(s_{t+1}, a) \\ & + \gamma^2 \sum_{a} \pi(a_{t+1}|s_{t+1}) \pi(a|s_{t+2}) Q_{t+1}(s_{t+2}, a). \end{aligned} \tag{7.11}$$

代入后展开，你会发现回报变成了这样几部分拼起来的：

- 第 1 步的真实奖励：r_{t+1}
- 第 2 步的真实奖励（要打折，且要受到第1步选择概率的约束）：$\gamma\pi(a_{t+1}|s_{t+1})r_{t+2}$
- 第 1 步错过的枝桠：$\gamma\sum_{a\neq a_{t+1}}\cdots$
- 第 2 步面临的所有选择（双重打折，且受第1步概率约束）：$\gamma^2\pi(a_{t+1}|s_{t+1})\sum_a\cdots$

3.N阶树状回报

$$g_{t:t+n} \triangleq r_{t+1} + \gamma \sum_{a\neq a_{t+1}} \pi(a|s_{t+1})Q_{t+n-1}(s_{t+1},a) + \gamma\pi(a_{t+1}|s_{t+1})g_{t+1:t+n}. \tag{7.12}$$

公式 7.13 把上面的递归公式一层一层“暴力”展开了

$$\begin{aligned} g_{t:t+n} = {} & r_{t+1} + \gamma\pi(a_{t+1}|s_{t+1})\cdot r_{t+2} + \cdots + \gamma^{n-1}\prod_{i=1}^{n-1}\pi(a_{t+i}|s_{t+i})\cdot r_{t+n} \\ & + \gamma \sum_{a\neq a_{t+1}} \pi(a|s_{t+1})Q_{t+1}(s_{t+1},a) \\ & + \cdots \\ & + \gamma^n \prod_{i=1}^{n-1}\pi(a_{t+i}|s_{t+i})\pi(a|s_{t+n})Q_{t+n-1}(s_{t+n},a). \end{aligned} \tag{7.13}$$

- **公式 7.12：** 就是把公式 7.10 里的下标 2 换成了 n，逻辑一模一样。把未来分成“错过的枝桠”和“继续延伸的主干”。
- **公式 7.13 (终极大公式)：** 这个就是把递归公式一层层暴力拆解到底的结果。连乘符号 $\prod_{i=1}^{n-1}\pi(\ldots)$ 的意思就是把一路上所有做过选择的概率全乘起来。

这棵树上任何一片“叶子”（某个未选中动作的 Q 值估算）想要对起点的最终目标值做出贡献，必须经过层层“扒皮”。它的权重等于**从起点走到它所经过的所有“边系数”的连乘**。

- 什么是“边系数”？就是 **打折率** γ $\times$ **走这条路的概率** $\pi(a|s)$。
  - 埋得越深（步数越长），乘的 γ 越多，影响越小。
  - 经过的冷门路径（π 极小）越多，最终权重越趋近于 0。

树状备份算法有一个巨大的优势：**它可以极其安全、自然地用于离策略 (Off-policy) 学习**。
因为在计算目标值时，主干路径的权重已经被目标策略 $\pi(a_t|s_t)$ 给重新加权（缩放）过了。即使你是用另一个完全随机的策略在环境中乱走（收集数据），只要你套用树状备份的公式，也能准确评估你的目标策略 π。这也是为什么公式里完全没有出现传统离策略算法中必须使用的“重要性采样 (Importance Sampling)”比例因子。

n步算法的比较与统一 (Comparison and Unification of n-step Algorithms)

我们已经学习了三种不同的 n 步自举算法：
1. **n步 Sarsa (n-step Sarsa):** 属于“完全采样”。每一步只看实际发生的那条路。
2. **n步期望 Sarsa (n-step Expected Sarsa):** 前面 $n-1$ 步看实际发生的路，最后一步评估所有可能发生的路（求期望）。
3. **n步树状备份 (n-step Tree backup):** 属于“完全期望”。每走一步，不仅看走过的路，还要评估路口所有没选的路（全树求期望）。
引出新问题： 既然这些算法各有千秋（有的方差大但计算快，有的计算慢但更稳定），我们在构建目标值 $g_{t:t+n}$ 时，能不能根据实际情况，**在“采样”和“期望”之间自由切换呢？**
为此，开发了一个大一统算法框架——$Q(\sigma)$ **算法**。

核心机制：神奇的“滑动变阻器” σ
$Q(\sigma)$ **算法的精髓在于引入了一个连续变量 σ_t。**

- **符号解析 σ_t (Sigma)：** 这是一个在时间步 t 取值范围为 $[0,1]$ 的控制参数。它就像一个用来调音的滑动变阻器。
  - **当 $\sigma_t = 1$ 时 (完全采样 / complete sample-based)：** 智能体在这一步完全模仿 Sarsa，只用实际走的那一条路径作为样本。
  - **当 $\sigma_t = 0$ 时 (完全期望 / pure expectation)：** 智能体在这一步完全模仿树状备份，考虑当前状态下所有可能动作的期望值。
  - **当 σ_t 处于 0 和 1 之间时：** 它是采样和期望的加权混合。

- **通俗举例：** 假设你在规划一条自驾游路线。
  • $\sigma = 1$ 就像是你**亲身去开了一遍**某条特定的路，然后用这次的真实体验来评估路况（纯体验派）。
  • $\sigma = 0$ 就像是你停在路口，**打开地图软件，把所有可能路线的预计时间都算了**

一遍，求了个平均值（纯理论派）。

• $Q(\sigma)$ 算法允许你在自驾游的第 1 天做"纯体验派"，第 2 天做"纯理论派"，随心所欲地动态调整。

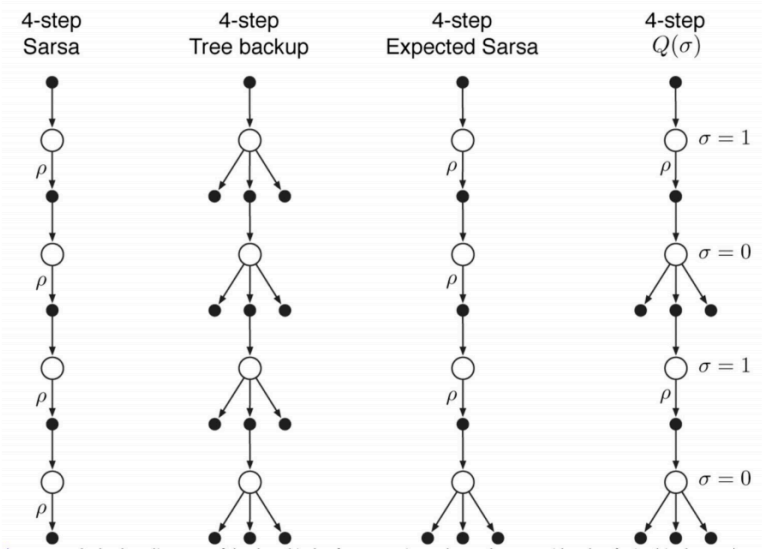

Figure 7.1: The backup diagrams for Sarsa, Tree backup, Expected Sarsa, and $Q(\sigma)$, under the setting of $n = 4$.

空心圆圈 ○ 代表**状态 (State)**。**实心黑点** ● 代表**动作 (Action)**

4步 Sarsa：在所有的 4 步里，它只盯着实际选中的那个动作一直走。没有任何旁枝末节，**全部都是采样 ($\sigma = 1$)**。

4步树状备份：沿途的每一步，它不仅看实际走的路，还会把所有没选的路也算进来。**全部都是期望 ($\sigma = 0$)**。

4步期望 Sarsa：前面老老实实地采样（$\sigma = 1$），只在最后提供最终估值的时候，对最后一步的所有动作求平均（$\sigma = 0$）。

4步 $Q(\sigma)$：第 1 步：$\sigma = 1$ (单抽样，不分叉)；第 2 步：$\sigma = 0$ (求期望，有分叉)；第 3 步：$\sigma = 1$ (单抽样，不分叉)；第 4 步：$\sigma = 0$ (求期望，有分叉)

标准 n步 Sarsa 和 n步期望 Sarsa的比较

标准 n步 Sarsa 的公式：

$$g_{t:t+n} = \underbrace{r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n}}_{\text{第一部分：前面 } n \text{ 步的真实奖励}} + \underbrace{\gamma^n Q_{t+n-1}(s_{t+n}, \mathbf{a_{t+n}})}_{\text{第二部分：最后一步的估值}}$$

n步期望 Sarsa (Expected Sarsa) 的公式：

$$g_{t:t+n} = \underbrace{r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n}}_{\text{第一部分：完全一样！}} + \underbrace{\gamma^n \sum_a \pi(a|s_{t+n}) Q_{t+n-1}(s_{t+n}, \mathbf{a})}_{\text{第二部分：变了！变成求期望}}$$

- **"前面看实际发生的路"：** 注意看两个公式的**第一部分**，它们是一模一样的！这里的 r_{t+1} 到 r_{t+n}，是你智能体在环境里**真实走出来的、脚踏实地拿到的奖励**。为了拿到这些真实的奖励，你的智能体沿途必须做出真实的动作（$a_t, a_{t+1} \ldots a_{t+n-1}$）。这就是所谓的"采样 (Sample)"，只看实际发生的那一条唯一路径。
- **"最后一步评估所有可能发生的路"：** 差异出现在**第二部分**（公式的尾巴，即自举的部分）。
  - 标准 Sarsa 在最后一步 s_{t+n} 时，是看你**实际抽签抽到了哪个动作** a_{t+n}，就拿这个特定动作的 Q 值来垫底。
  - 期望 Sarsa 在最后一步 s_{t+n} 时，**把骰子扔掉**了。它不看你实际准备走哪一步，而是把这个路口**所有可能的动作** a 的 Q 值都拿出来，按照策略概率 π 算了一个加权平均数（求期望 $\sum$）。

假设你在玩一个原神，要评估"进入恶龙洞穴"这个决定的好坏，你决定往前推演 3 步（$n = 3$）：

• **前面 2 步（实际发生）：** 你真刀真枪地走进了洞穴（第1步），并且真实地躲过了一个陷阱（第2步），你把这两步掉落的真实金币记在了账上。

• **最后第 3 步（面对恶龙，求期望）：** 现在你站在了恶龙面前（状态 s_{t+n}）。你有三个技能：50%概率用剑砍，40%概率用魔法，10%概率逃跑。

- 如果用 **标准 Sarsa**：你可能刚好脑抽，随机抽到了"逃跑"（虽然概率只有10%），于是你用"逃跑"的极低收益作为最后一步的估值。这会让整条路线的评估大打折扣（被最后一步的运气给坑了）。
- 如果用 **期望 Sarsa**：你不实际抽签。你算一笔稳当账：`50% * 剑的收益 + 40% * 魔法收益 + 10% * 逃跑收益`。拿这个**平均期望值**作为最后的估值底座。

难懂概念解释：

公式里的 g（或者叫 $g_{t:t+n}$）。
它由两部分强行拼凑而成：
• **前面 n 步的真实体验：** 从时间点 t 走到 $t+n$，你确确实实捡到了几块金币（拿到了 $r_{t+1}, r_{t+2} \ldots r_{t+n}$）。这些是**真金白银**，加上时间折扣 γ 后累加。
• **第 n 步的脑补估计：** 到了状态 S_{t+n}，你不想往后走了，于是你看了眼手里的“价值字典”，查了一下当前位置的估值 $V(S_{t+n})$，打个折（γ^n）加进去。
这就叫**自举 (Bootstrapping)**：用一部分真实数据，加上一部分自己的主观猜测，拼成一个完整的目标 g。

经典的“增量更新法则”：$V(s) \leftarrow V(s) + \alpha(g - V(s))$
• $V(s)$：你旧的认知（比如你原来以为 S_0 值 10 分）。
• g：你刚算出来的、带着新鲜真实体验的新目标（比如新算出来是 15 分）。
• $g - V(s)$：差距（误差是 +5 分）。
• α：学习率（比如 0.1）。你不会一口气把 S_0 改成 15 分，而是朝着目标迈一小步，变成 $10 + 0.1 \times 5 = 10.5$ 分。

为什么到 S_3 了才更新 S_0？
因为你没有“预知未来”的超能力！
假设 $n = 3$。你想更新 S_0 的价值，你需要从 S_0 出发，往后走 3 步的真实金币（r_1, r_2, r_3）。
• 当你在 S_0 时，未来还没发生，你拿不到金币。
• 当你在 S_1 时，你只有 r_1。
• 当你在 S_2 时，你只有 r_1, r_2。
• **只有当你双脚踏进 S_3 的那一刻**，环境才刚刚把 r_3 发给你。这时候你才终于凑齐了计算 g 所有的材料！所以你只能在这个时候，回过头去把 S_0 这笔旧账给结了。

那到 S_4 了更新 S_1 的 V？怎么算？
这就是一个大小为 n 的“滑动窗口”。
我们把时间线拉开，仔细算一下当走到 S_4 时，更新 S_1 的全过程（假设 $n = 3$）：
• **当前所处位置：** 你刚刚执行完动作 A_3，拿到了奖励 R_4，双脚刚刚踩进状态 S_4。

- **准备结账的对象：** 往前推 3 步，现在该给 S_1 结账了。
- **滑动窗口里的数据：** 你的窗口框住了从 S_1 出发后的 3 步真实奖励，也就是 R_2, R_3, R_4。（注意：没有 R_1，因为 R_1 是从 S_0 到 S_1 的奖励，属于上一个阶段的历史了）。
- **计算目标值 g：** $g = R_2 + \gamma R_3 + \gamma^2 R_4 + \gamma^3 V(S_4)$
（你看，真金白银是 R_2, R_3, R_4，估计值是你当前所处的 S_4 的 V 值）。
- **执行更新：** $V(S_1) \leftarrow V(S_1) + \alpha(g - V(S_1))$

$Q_t(s,a)$ 意思是：在时间走到 t 时刻，你脑子里对世界认知的**第 t 版快照**。
$Q_{t+1}(s,a)$ 意思是：你的认知经过了一步更新后，升级到了**第 $t+1$ 版快照**。

为什么在 2 步树状回报里，全都是 Q_{t+1}？
我们把 2 步树状回报的计算过程当成一次**“两天一夜的实地考察”**。
- **起点：** 时间 t，你在状态 S_t。
- **第一天：** 你走到 S_{t+1}，拿到奖励 r_{t+1}。
- **第二天：** 你走到 S_{t+2}，拿到奖励 r_{t+2}。考察结束，开始算总账！

现在，你要算 2 步目标值 $g_{t:t+2}$。根据树状备份的规则，你需要对两处“枝桠”进行估值：
1. **第一天错过的枝桠：** 在 S_{t+1} 路口没选的动作。
2. **第二天面临的枝桠：** 在 S_{t+2} 路口未来的所有动作。

重点来了：
当你坐在书桌前算这笔总账时，你应该用哪一版的“估价指南”（Q 表）去给这些枝桠标价呢？
书里的严谨数学设定是：**对于 n 步自举，统一使用 $t+n-1$ 时刻的认知版本。**
因为这里是 2 步（$n=2$），所以 $t+n-1 = t+2-1 = \mathbf{t+1}$。
这就是为什么：
- 在算第一天错过的枝桠时，公式写的是 $Q_{t+1}(s_{t+1}, a)$。
- 在算第二天面临的枝桠（即展开的 $g_{t+1:t+2}$ 里），用的**依然是** $Q_{t+1}(s_{t+2}, a)$。

三、 这么设计有什么深刻含义？（通俗解释）
1. 评估标准必须“一碗水端平”（保持一致性）：
你在算这一笔完整的 2 步账单时，不论是评估第一天路过的风景，还是评估第二天终点的风景，你必须拿**同一版指南**（Q_{t+1} 快照）去打分。如果你在公式的前半段用旧版本，后半段用新版本，你的目标值 g 就会发生内部逻辑的扭曲。
2. 为什么不用最新的 Q_{t+2} 而是退一格用 Q_{t+1}？
这涉及到一个非常底层的代码实现细节：因为你要更新的是 S_t 的价值，而当时间真正走到 $t+2$ 时，你刚好要开始准备更新 S_t 了。在这次更新发生**之前**，你手里握着的最新、最稳定的全局认知版本，其实是上一步（也就是 $t+1$ 步）留下的那一版。所以书里严谨地把它标记为了 Q_{t+1}。